CodeSense

CI/CD Integration Guide — Azure DevOps

This guide walks through triggering CodeSense scans automatically from an Azure DevOps pipeline, using a project-scoped API key instead of a human login. It reuses the same scan endpoints the web application itself uses — there is no separate CI-only report format, and results always land in the application's normal scan history for the target project.

1. Generate an API Key

In the CodeSense web application, open the target project, go to the **Settings** tab, then **CI API Keys**, and select **Generate new key**. Copy the plaintext value shown immediately — it is not shown again after you navigate away.

> **Scope:** an API key can only submit new scans. It cannot read scan results, list projects, or perform any other action — a key that leaks in a CI log or misconfigured pipeline cannot be used to pull data out of CodeSense.

2. Configure Pipeline Variables

Store the following as **secret** pipeline variables in Azure DevOps (Pipeline → Edit → Variables). Never place the key value directly in the YAML file.

- **CODESENSE_API_URL** — the base URL of your reachable CodeSense instance (e.g. `http://10.0.4.21`). Leave unset if the application is not reachable from this pipeline's agents — see Section 5.
- **CODESENSE_API_KEY** — the generated key from Step 1 (begins with `csk_`).
- **CODESENSE_PROJECT_ID** — the project ID the key is bound to, shown next to the key in the CI API Keys panel.

3. Recommended — Zip Upload Mode

This mode packages the pipeline's exact checkout and uploads it directly — it scans precisely what the pipeline built, not a separate clone. Add the following steps to your pipeline YAML:

```
steps:
  - task: PowerShell@2
    displayName: 'Package workspace for CodeSense scan'
    inputs:
      targetType: inline
      script: |
        Compress-Archive -Path "$(Build.SourcesDirectory)\*" `
          -DestinationPath "$(Build.ArtifactStagingDirectory)\codesense-scan.zip" -Force
```

```
- task: PublishPipelineArtifact@1
  displayName: 'Publish scan payload as a pipeline artifact'
  inputs:
    targetPath: '$(Build.ArtifactStagingDirectory)\codesense-scan.zip'
    artifact: 'codesense-scan-payload'

- task: PowerShell@2
  displayName: 'Submit scan to CodeSense (if reachable)'
  condition: ne(variables['CODESENSE_API_URL'], '')
  inputs:
    targetType: inline
    script: |
      $ErrorActionPreference = 'Stop'
      $zipPath = "$(Build.ArtifactStagingDirectory)\codesense-scan.zip"
      $uri     = "$(CODESENSE_API_URL)/api/scans/create/"

      # Windows PowerShell 5.1 does NOT support Invoke-RestMethod's -Form
      # parameter (added in PowerShell 6.1+) -- this builds the multipart
      # upload manually so it works on standard Windows build agents.
      Add-Type -AssemblyName System.Net.Http

      $httpClient = New-Object System.Net.Http.HttpClient
      $httpClient.DefaultRequestHeaders.Authorization = `
        New-Object System.Net.Http.Headers.AuthenticationHeaderValue(
          "Bearer", "$(CODESENSE_API_KEY)")

      $form = New-Object System.Net.Http.MultipartFormDataContent
      $form.Add((New-Object System.Net.Http.StringContent(
        "$(Build.Repository.Name)-$(Build.SourceBranchName)")), "scan_name")
      $form.Add((New-Object System.Net.Http.StringContent(
        "$(CODESENSE_PROJECT_ID)")), "project_id")

      $fileStream  = [System.IO.File]::OpenRead($zipPath)
      $fileContent = New-Object System.Net.Http.StreamContent($fileStream)
      $fileContent.Headers.ContentType = `
        [System.Net.Http.Headers.MediaTypeHeaderValue]::Parse("application/zip")
      $form.Add($fileContent, "zip_file", [System.IO.Path]::GetFileName($zipPath))

      try {
        $response = $httpClient.PostAsync($uri, $form).GetAwaiter().GetResult()
        $body = $response.Content.ReadAsStringAsync().GetAwaiter().GetResult()
        if (-not $response.IsSuccessStatusCode) {
          Write-Error "CodeSense scan submission failed: $($response.StatusCode) $body"
          exit 1
        }
        Write-Host "CodeSense scan submitted: $body"
      } finally {
        $fileStream.Dispose()
        $form.Dispose()
        $httpClient.Dispose()
      }
```

The artifact-publish step always runs and never fails for network reasons. The submission step only runs when `CODESENSE_API_URL` is set, and fails the pipeline step (non-zero exit) on any non-2xx response — a revoked key, wrong project, or unreachable application all surface as a visible pipeline failure rather than silently doing nothing.

> **Note:** this is a submission check only, not a build gate. A scan that submits successfully and later reports findings does not fail the build on its own.

4. Alternative — Repository Clone Mode

Instead of uploading a zip, CodeSense can clone the repository itself. This requires the CodeSense instance to have network access to your git host, and scans a fresh clone of the pushed branch rather than the pipeline's exact checkout.

```
- task: PowerShell@2
  displayName: 'Trigger CodeSense scan via repo clone'
  inputs:
    targetType: inline
    script: |
      $ErrorActionPreference = 'Stop'
      $uri  = "$(CODESENSE_API_URL)/api/scans/github/"
      $body = @{
        token      = "$(CODESENSE_GIT_TOKEN)"
        username   = "$(CODESENSE_GIT_OWNER)"
        repo       = "$(Build.Repository.Name)"
        branch     = "$(Build.SourceBranchName)"
        project_id = "$(CODESENSE_PROJECT_ID)"
      } | ConvertTo-Json

      try {
        $response = Invoke-RestMethod -Uri $uri -Method Post -Body $body `
          -ContentType 'application/json' `
          -Headers @{ Authorization = "Bearer $(CODESENSE_API_KEY)" }
        Write-Host "CodeSense scan submitted: $($response | ConvertTo-Json -Compress)"
      } catch {
        Write-Error "CodeSense scan submission failed: $($_.Exception.Message)"
        exit 1
      }
```

This step uses a plain JSON body, not a file upload, so the standard `Invoke-RestMethod` cmdlet works unmodified on Windows PowerShell 5.1 here.

5. If CodeSense Isn't Reachable From CI

Leave `CODESENSE_API_URL` unset. The submission step is skipped automatically and only the artifact-publish step runs, with no network dependency. The `codesense-scan-payload` artifact stays available on the pipeline run. Later, from anywhere that can reach the application, download that artifact and submit it using the same script by hand or via a scheduled job — it is the same `/api/scans/create/` call, just made later from a different place. No separate import endpoint exists.

6. Storing and Rotating the Key

The API key is a long-lived credential with no automatic expiry.

- Store it only in Azure DevOps's secret pipeline variables — never in the YAML file itself, never logged.

- If any pipeline step's output could echo the key (for example, verbose logging), mark the variable as secret so Azure DevOps redacts it automatically.
- Revoke and regenerate a key from the same **Settings** → **CI API Keys** panel if it is ever suspected of being exposed. Revocation is immediate.

Troubleshooting: Common Responses

Status	Meaning
`202 Accepted`	Scan submitted successfully and queued.
`401 Unauthorized`	The API key is invalid, mistyped, or has been revoked.
`403 Permission denied`	The key was used against an endpoint other than the four scan-creation endpoints. CI keys cannot read results or manage projects — this is expected behavior, not a misconfiguration to work around.
`403 Trial limit reached`	The account's scan allowance for this billing period has been used. Contact your CodeSense representative to upgrade.
`409 Conflict`	Another scan is already running on this instance. CodeSense processes one scan at a time; retry after the current scan finishes.

Questions about this integration? Contact your CodeSense delivery team.